

DATA 570



Recap of DATA 570: Predictive Modelling

DATA 570: Predictive Modelling

AUTHOR

Dr. Irene Vrbik

AFFILIATION

Masters of Data Science UBCO

Introduction

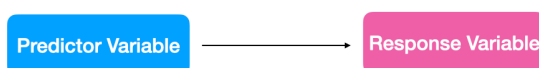
In this lecture we will recap the topics we covered during the DATA 570 module.

Key Concepts of Regression

What: Regression analysis is a fundamental statistical technique used to examine the relationship between one **response** (aka *dependent*, *target*) variable and one or more **predictor** variables (aka *independent variables* or *features*).

- Y (**response** variable):
 - this is a *continuous* random variable and the thing we are trying to predict
 - e.g. house prices, blood pressure, age of a mollusc,
- X (**predictor(s)**): these are usually denoted $X_1, X_2, \dots, X_p$, so p denotes.
 - They are the factors you believe have an impact on the dependent variable. They can be *categorical* or *numeric*.
 - e.g. size of the house, age of the patient, shell weight of a mollusc

Goal: Its primary goal is to model and understand how changes in the predictor variables influence the response variable.



Depending on which of the two we want to do, however, we might land on different models:

- **Prediction:** Forecasting future values based on historical data. For example, predicting future sales based on past performance.

- **Inference:** Understanding the strength and type of relationships between variables. For example, assessing how education level affects income (in this case we might prioritize more simpler interpretable models)

Where: This method is widely used in various fields such as economics, biology, engineering, social sciences, and machine learning for both prediction and inference.

Types of regression models

We will go through the different regression models we talked about one-by-one. Below is a summary of their individual Pros and Cons.

Model Type	Pros	Cons
Simple Linear Regression	- Easy to understand and interpret - Quick to implement - Low risk of overfitting - Serves as a good baseline model	- Limited to one predictor - Assumes linearity - Sensitive to outliers - Limited predictive power
Multiple Linear Regression	- Enhanced predictive power with multiple predictors - Controls for confounding variables - Flexibility to model interactions - Facilitates hypothesis testing	- Susceptible to multicollinearity - Risk of overfitting with too many predictors - Complex interpretation with many variables - Requires larger sample sizes
Polynomial Regression	- Models nonlinear relationships - Improved fit for curved data - Relatively easy to implement - Helps understand varying rates of change	- Prone to overfitting with high-degree terms - Multicollinearity among polynomial terms - Less intuitive interpretation - Risky extrapolation
KNN	- Easy to understand and implement - no data assumptions - handles complex non-linear structures	- Less interpretable - significant computation - must normalize features - Prone to underfitting (resp. overfitting) for large (resp. large) k - Performance deteriorates as p increases

Choosing the right regression model

1. **Visual Diagnostics to assess linearity:** Check the residual vs. fitted plots to assess if the model is appropriate
 - if yes, the residuals should be randomly scattered around 0, with no clear pattern, curvature, or systematic spread.
 - If you see a pattern (like a curve), it can indicate that the model is underspecified and needs adjustment or that a nonlinear model should be used.

2. **Compare Models using metrics:** Adjusted R-squared, test MSE, p-values, AIC and methods like stepwise and best subsets regression help refine models. Each has limitations, especially with complex datasets.
3. **Consider model complexity and goals:** All other factors being equal, choose the simplest model that provides an adequate fit to avoid overfitting and ensure better generalizability.

Goals: The “best” model depends on your goals.

- **For prediction:** focus on metrics like test MSE
- **For inference:** A parsimonious model with fewer variables may be preferable.

Real-World Complications

Issues like multicollinearity, outliers, and omitted variable bias, can distort results, emphasizing the need for careful model choice.

Practical Tip

Emphasize theory over data-driven decisions, aim for simplicity, and validate with residual plots. A balanced, theory-driven approach aids in avoiding overfitting.

Linear Regression

Simple Linear Regression: Examines the relationship between one dependent variable and one independent variable using a straight-line (linear) relationship.

$$Y = \beta_0 + \beta X + \epsilon$$

Shark Attacks Example

For instance, you can download the [shark_attacks.csv](#) data

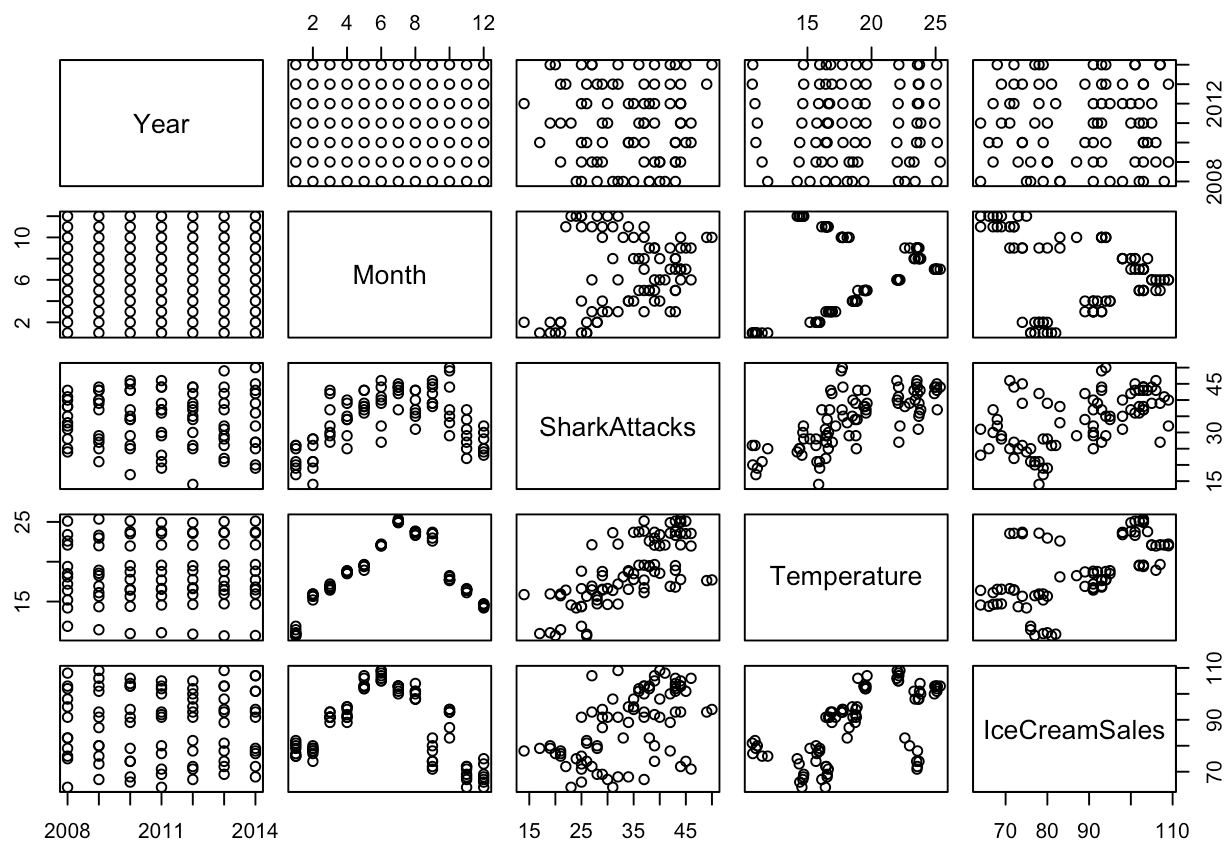
```
df <- read.csv("data/shark_attacks.csv")
df
```

Year	Month	SharkAttacks	Temperature	IceCreamSales
<int>	<int>	<int>	<dbl>	<int>
2008	1	25	11.90000	76
2008	2	28	15.20000	79
2008	3	32	17.20000	91
2008	4	35	18.50000	95
2008	5	38	19.40000	103
2008	6	41	22.10000	108
2008	7	43	25.10000	102
2008	8	40	23.40000	98
2008	9	38	22.60000	83
2008	10	33	18.10000	83

1-10 of 84 rows

Previous **1** [2](#) [3](#) [4](#) [5](#) [6](#) ... [9](#) [Next](#)

Plot the data:

`plot(df)`

It looks like there's high correlation between average monthly temperatures and shark attacks, and also between ice cream consumption and monthly temperatures, so let's add the **Temperature** variable to our model.

Fitting a SLR:

```
slr <- lm(SharkAttacks~IceCreamSales, data = df)
summary(slr)
```

Call:

```
lm(formula = SharkAttacks ~ IceCreamSales, data = df)
```

Residuals:

Min	1Q	Median	3Q	Max
-16.5813	-4.6829	-0.9704	4.8391	17.7726

Coefficients:

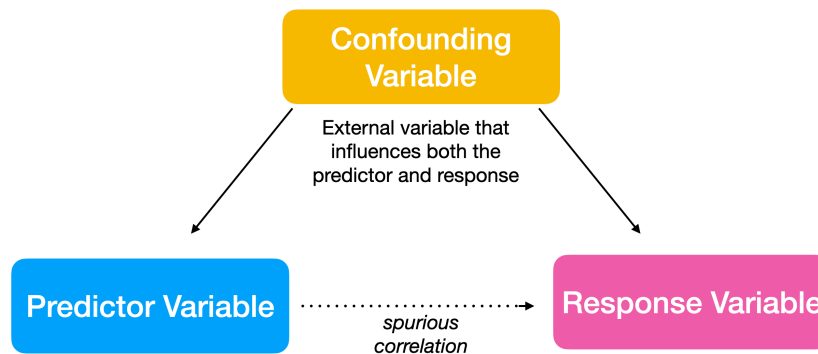
	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	4.35231	5.23776	0.831	0.408
IceCreamSales	0.33627	0.05874	5.725	1.65e-07 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 7.173 on 82 degrees of freedom
 Multiple R-squared: 0.2855, Adjusted R-squared: 0.2768
 F-statistic: 32.77 on 1 and 82 DF, p-value: 1.648e-07

The model suggests that there is a statistically significant positive association between **IceCreamSales** and **SharkAttacks**. However, **correlation does not imply causation**; ice cream sales likely do not cause shark attacks directly, but rather both may be driven by an underlying variable like temperature or season.

What we're seeing here is the workings of a **confounding variable**. To elaborate **IceCreamSales** is related to both the predictor (**SharkAttacks**) and another response variable (in this case **Temperature**). Hence and created a misleading association between them.



Notice that once that confounding variable (**Temperature**) is “controlled for” in the model, **IceCreamSales** is no longer significant.

```
m1r <- lm(SharkAttacks~IceCreamSales+Temperature, data = df)
summary(m1r)
```

Call:

```
lm(formula = SharkAttacks ~ IceCreamSales + Temperature, data = df)
```

Residuals:

Min	1Q	Median	3Q	Max
-15.2433	-2.6065	-0.0976	2.8733	16.6965

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	0.56510	4.30514	0.131	0.8959
IceCreamSales	0.10459	0.05957	1.756	0.0829 .
Temperature	1.29276	0.19802	6.528	5.4e-09 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 5.842 on 81 degrees of freedom
 Multiple R-squared: 0.5319, Adjusted R-squared: 0.5203
 F-statistic: 46.01 on 2 and 81 DF, p-value: 4.469e-14

Multiple Linear Regression: Involves two or more independent variables to predict the dependent variable.

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_p X_p + \epsilon$$

Assumptions:

- Linearity: The relationship between predictors and the outcome is linear.
- Independence: Observations are independent of each other.
- Homoscedasticity: Constant variance of errors.
- Normality: Errors are normally distributed.

Interaction models

Interaction effects occur when the effect of one variable on a dependent variable depends on the value of another variable.

With categorical variables

Consider a taste test evaluating enjoyment based on different foods (e.g., ice cream and hot dogs) and condiments (e.g., chocolate sauce and mustard). The enjoyment derived from a condiment depends on the type of food, illustrating an interaction effect. Analyzing such data without accounting for interaction effects can lead to misleading conclusions.

```
food <- read.csv("data/Interactions_Categorical.csv")
str(food)
```

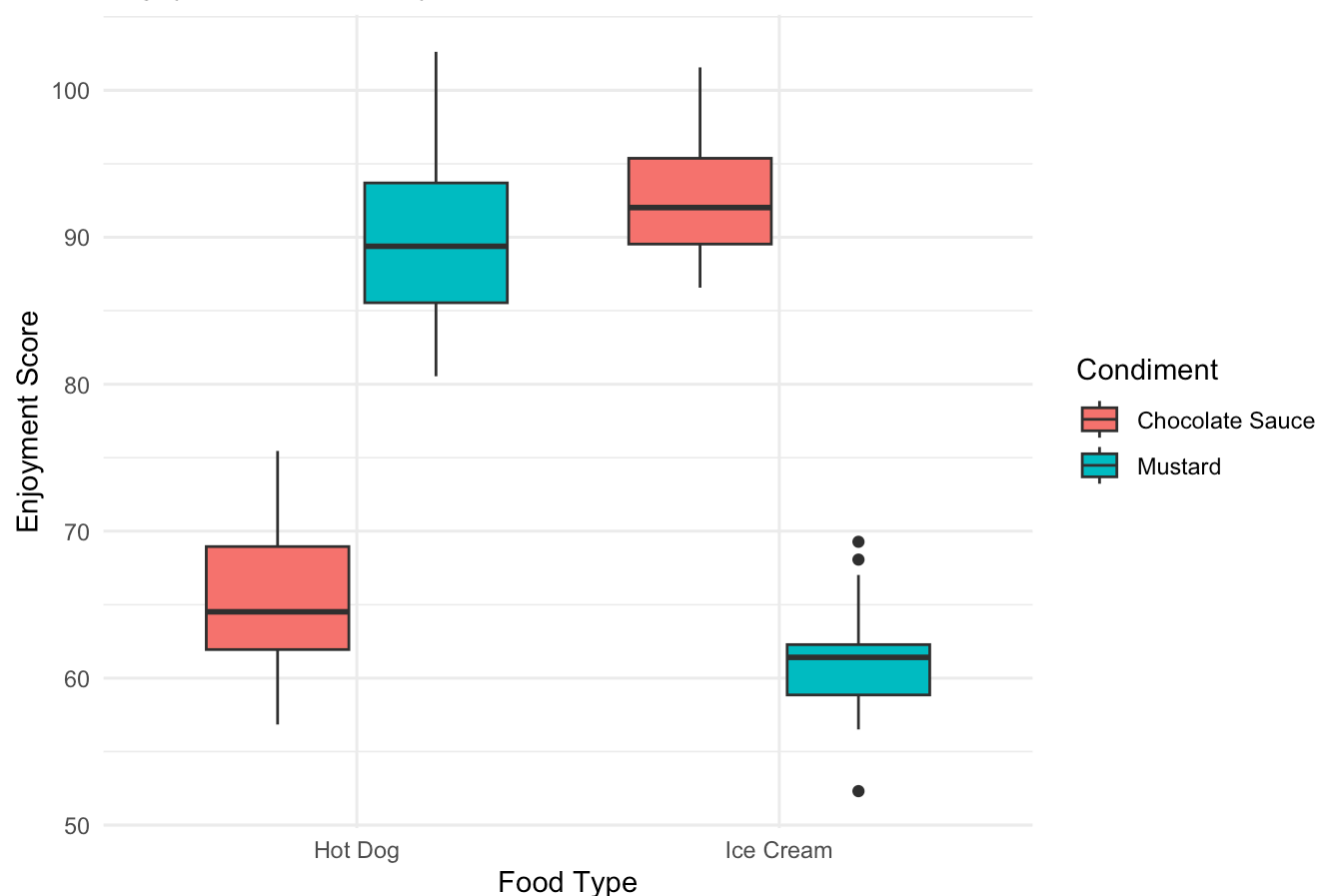
```
'data.frame':  80 obs. of  3 variables:
 $ Enjoyment: num  81.9 84.9 90.3 89.6 97.7 ...
 $ Food      : chr  "Hot Dog" "Hot Dog" "Hot Dog" "Hot Dog" ...
 $ Condiment: chr  "Mustard" "Mustard" "Mustard" "Mustard" ...
```

```
food$Food <- factor(food$Food)
food$Condiment <- factor(food$Condiment)
```

One way to explore your data when both of your variables are categorical is through box plots

► Code

Enjoyment Scores by Food and Condiment



Let's fit the interaction model:

$$\text{Enjoyment} = \beta_0 + \beta_1 \text{Food} + \beta_2 \text{Condiment} + \beta_3 \text{Food} \cdot \text{Condiment}$$

```
int_mod <- lm(Enjoyment ~ Food*Condiment, data = food)
summary(int_mod)
```

Call:

```
lm(formula = Enjoyment ~ Food * Condiment, data = food)
```

Residuals:

Min	1Q	Median	3Q	Max
-9.068	-3.068	-0.407	2.802	13.015

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	65.317	1.120	58.34	<2e-16 ***
FoodIce Cream	27.731	1.583	17.52	<2e-16 ***
CondimentMustard	24.289	1.583	15.34	<2e-16 ***
FoodIce Cream:CondimentMustard	-56.028	2.239	-25.02	<2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 5.007 on 76 degrees of freedom
 Multiple R-squared: 0.8935, Adjusted R-squared: 0.8892
 F-statistic: 212.4 on 3 and 76 DF, p-value: < 2.2e-16

Like before we would simply have cases for each level:

```
betas = coef(int_mod)
```

$$\begin{aligned}
 \text{Enjoyment} &= \beta_0 + \beta_1 \text{IceCream} + \beta_2 \text{Mustard} + \beta_3 \text{IceCream} \cdot \text{Mustard} \\
 &= \begin{cases} \beta_0 + \beta_1 \cdot (0) + \beta_2 \cdot (1) & = \text{if Hotdog with Mustard} \\ \beta_0 + \beta_1 \cdot (0) + \beta_2 \cdot (0) & = \text{if Hot Dog with Chocolate Sauce} \\ \beta_0 + \beta_1 \cdot (1) + \beta_2 \cdot (1) + \beta_3 \cdot (1) \cdot (1) & = \text{if Ice Cream with Mustard} \\ \beta_0 + \beta_1 \cdot (1) + \beta_2 \cdot (0) + \beta_3 \cdot (1) \cdot (0) & = \text{if Ice Cream with Chocolate Sauce} \end{cases} \\
 &= \begin{cases} \beta_0 + \beta_2 & = \text{if Hotdog with Mustard} \\ \beta_0 & = \text{if Hot Dog with Chocolate Sauce} \\ \beta_0 + \beta_1 + \beta_2 + \beta_3 & = \text{if Ice Cream with Mustard} \\ \beta_0 + \beta_1 & = \text{if Ice Cream with Chocolate Sauce} \end{cases} \\
 &= \begin{cases} 89.61 & = \text{if Hotdog with Mustard} \\ 65.32 & = \text{if Hot Dog with Chocolate Sauce} \\ 61.31 & = \text{if Ice Cream with Mustard} \\ 93.05 & = \text{if Ice Cream with Chocolate Sauce} \end{cases}
 \end{aligned}$$

```
'data.frame': 29 obs. of 4 variables:
 $ Strength : num 100.7 97.8 88.4 85.4 93.2 ...
 $ Temperature: num 100.6 109.5 104 99.4 107.2 ...
 $ Pressure : num 81.1 72.4 74.6 65 67.4 ...
 $ Time : num 33.3 32.9 35.3 35 32.8 ...
```

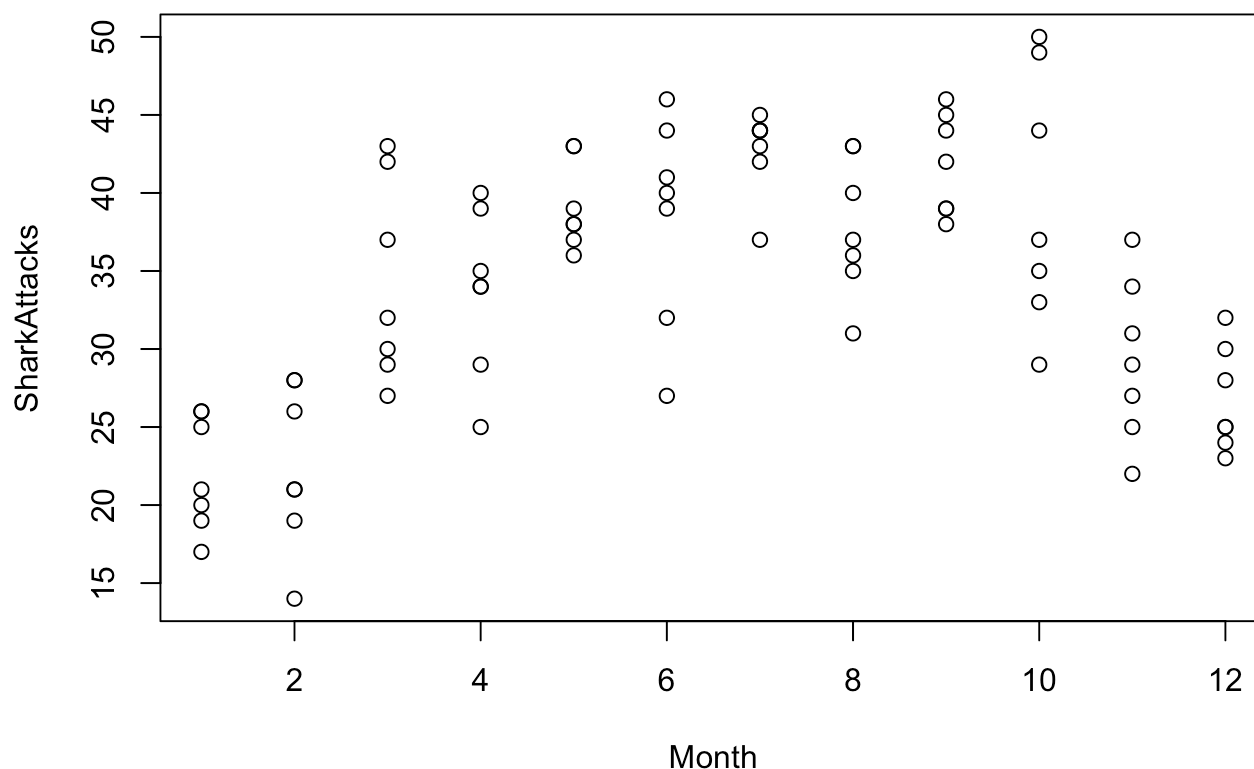
Polynomial Regression

Polynomial Regression: Extends linear models by adding polynomial terms of the predictors to capture nonlinear relationships. The M th order polynomial regression model is given by:

$$Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \cdots + \beta_M X^M + \epsilon$$

For instance, if we return to the [Shark Attacks Example](#), we can see that the relationship between attacks and months is non-linear; that is, there appear to be more in the summer months and dips outside of summertime.

```
plot(SharkAttacks~Month, data = df)
```

To accommodate this in R we'd fit a polynomial regression model. We probably only need a quadratic, which can accommodate one “bend”.

```
order2 <- lm(SharkAttacks~poly(Month,2), data = df)
summary(order2)
```

Call:

```
lm(formula = SharkAttacks ~ poly(Month, 2), data = df)
```

Residuals:

Min	1Q	Median	3Q	Max
-13.4006	-3.1150	-0.0599	3.1404	13.8463

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	34.0000	0.5867	57.952	< 2e-16 ***
poly(Month, 2)1	21.2083	5.3771	3.944	0.000169 ***
poly(Month, 2)2	-55.8053	5.3771	-10.378	< 2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 5.377 on 81 degrees of freedom

Multiple R-squared: 0.6035, Adjusted R-squared: 0.5937
 F-statistic: 61.63 on 2 and 81 DF, p-value: < 2.2e-16

For fun let's compare this with an order-3 polynomial. In this case, the comparison is easy to make visually:

► Code

Call:

```
lm(formula = SharkAttacks ~ poly(Month, 3), data = df)
```

Residuals:

Min	1Q	Median	3Q	Max
-13.1489	-3.0147	0.0102	3.7208	13.0911

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	34.0000	0.5844	58.175	< 2e-16 ***
poly(Month, 3)1	21.2083	5.3565	3.959	0.000162 ***
poly(Month, 3)2	-55.8053	5.3565	-10.418	< 2e-16 ***
poly(Month, 3)3	-6.8271	5.3565	-1.275	0.206160

 Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 5.356 on 80 degrees of freedom
 Multiple R-squared: 0.6113, Adjusted R-squared: 0.5968
 F-statistic: 41.95 on 3 and 80 DF, p-value: < 2.2e-16

► Code

Call:

```
lm(formula = SharkAttacks ~ poly(Month, 3), data = df)
```

Residuals:

Min	1Q	Median	3Q	Max
-13.1489	-3.0147	0.0102	3.7208	13.0911

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	34.0000	0.5844	58.175	< 2e-16 ***
poly(Month, 3)1	21.2083	5.3565	3.959	0.000162 ***
poly(Month, 3)2	-55.8053	5.3565	-10.418	< 2e-16 ***
poly(Month, 3)3	-6.8271	5.3565	-1.275	0.206160

 Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 5.356 on 80 degrees of freedom
 Multiple R-squared: 0.6113, Adjusted R-squared: 0.5968
 F-statistic: 41.95 on 3 and 80 DF, p-value: < 2.2e-16

► Code

Call:

```
lm(formula = SharkAttacks ~ poly(Month, 3), data = df)
```

Residuals:

	Min	1Q	Median	3Q	Max
	-13.1489	-3.0147	0.0102	3.7208	13.0911

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	34.0000	0.5844	58.175	< 2e-16 ***
poly(Month, 3)1	21.2083	5.3565	3.959	0.000162 ***
poly(Month, 3)2	-55.8053	5.3565	-10.418	< 2e-16 ***
poly(Month, 3)3	-6.8271	5.3565	-1.275	0.206160

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

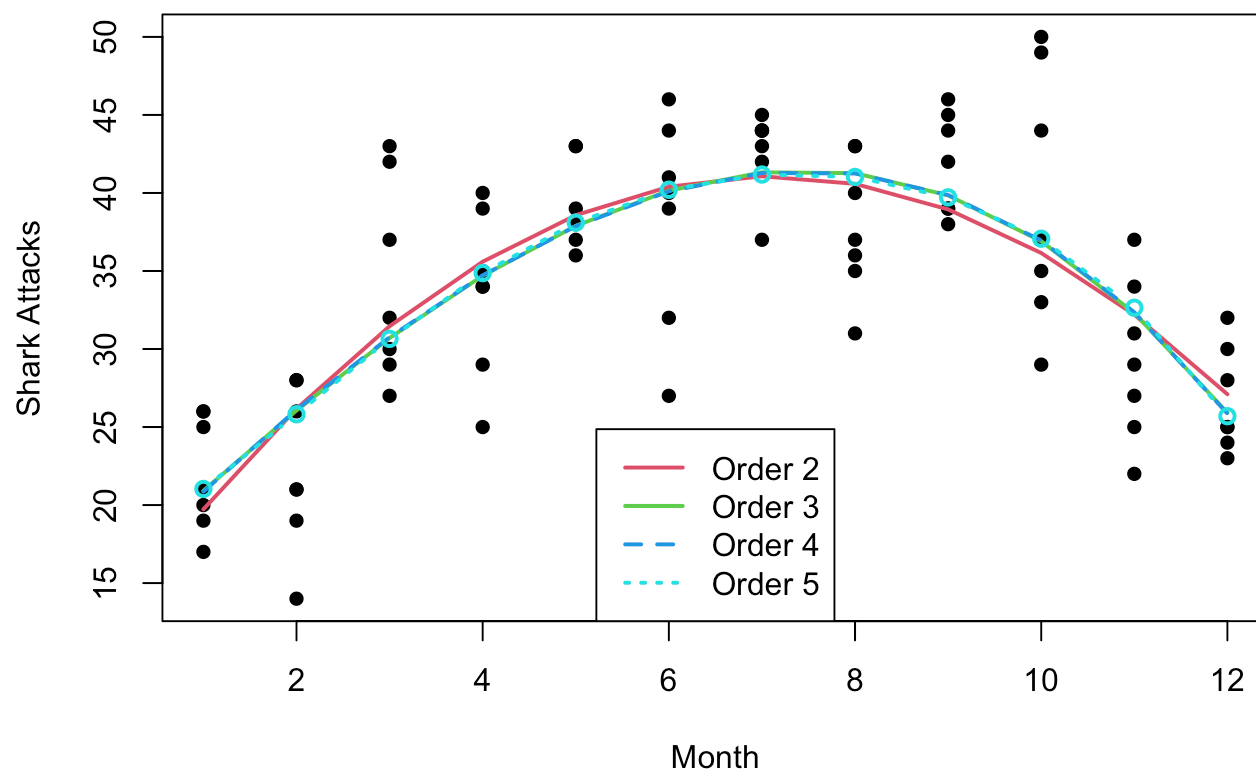
Residual standard error: 5.356 on 80 degrees of freedom

Multiple R-squared: 0.6113, Adjusted R-squared: 0.5968

F-statistic: 41.95 on 3 and 80 DF, p-value: < 2.2e-16

► Code

Polynomial Regression Fits



Model	Adjusted_R2	AIC	BIC
Order 2	0.5936670	525.9281	535.6514
Order 3	0.5967756	526.2395	538.3936
Order 4	0.5916955	528.2346	542.8195
Order 5	0.5869809	530.1289	547.1446

The models exhibit only slight differences in adjusted R^2 , and show comparable fits on the scatterplot. Given that these models capture the data's underlying pattern equally well, in the principle of parsimony, we prefer the simpler, lower-order quadratic (Order 2) model since higher-order polynomials, such as the cubic (Order 3) and quartic (Order 4) are likely to add unnecessary complexity without meaningful gains in explanatory power (i.e. risk overfitting).

Furthermore, the quadratic (Order 2) polynomial obtained the smallest AIC and BIC value which indicates that it strikes the best balance between accuracy and simplicity.

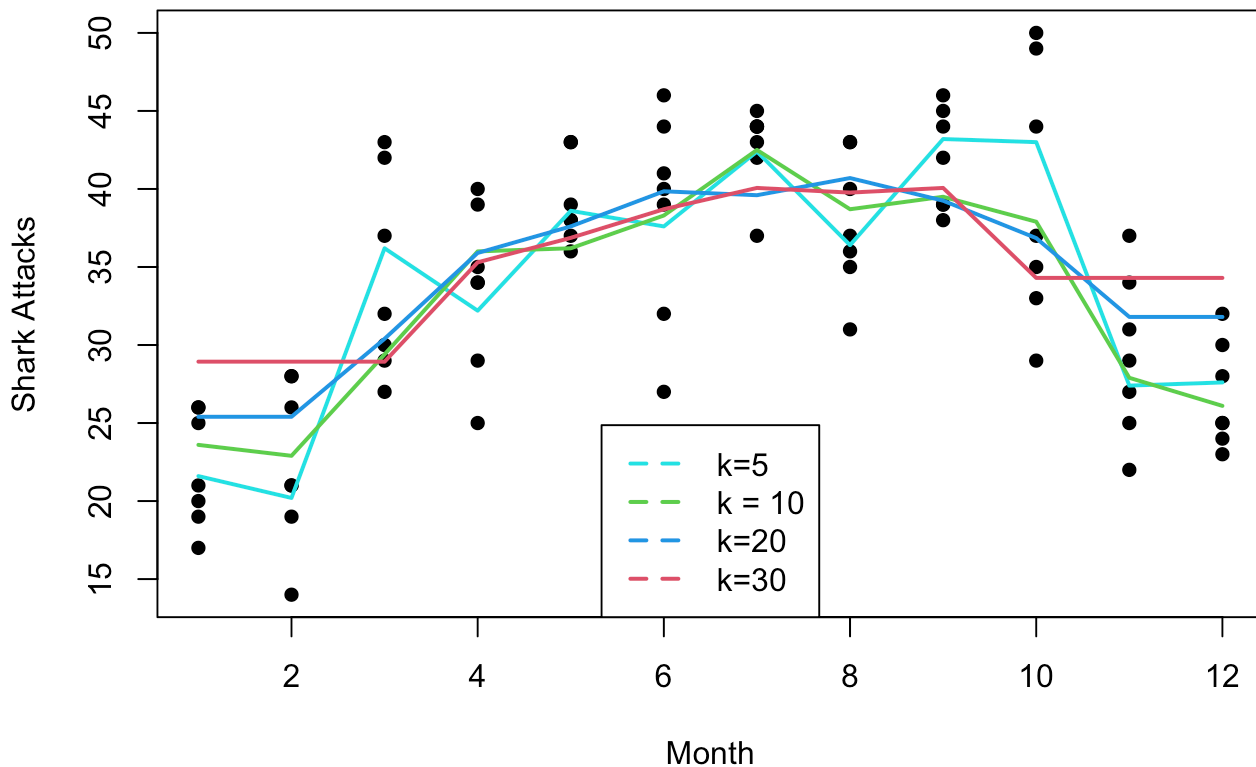
Note: While we could have split the data into training and testing sets to evaluate model performance on unseen data via MSE_{test} , our sample size is limited to 84 points. With such a small dataset, training and testing splits might reduce our ability to detect meaningful patterns or introduce variability in our results.

KNN Regression

K-Nearest Neighbors Regression (KNN Regression): This is a nonparametric approach which predicts the value of a response variable based on the average (or weighted average) of the y values of the k nearest neighbors in the feature space.

► Code

KNN Regression Fit for Shark Attacks (k = 5)



Here we can just plot the different fits and choose the k that fits the data the closest, in this case, is probably $k = 20$. Again we could use the test/train split (validation approach) to assess the different values of k , but with a small a data set as this one, I opted to keep all data points for fitting.

Foreshadow: Next model we'll see how we can get this using cross-validation (which is the most common way you'll see turning parameters being picked in predictive modeling).

Assessing Regression models

Goodness of fit

- The **Mean Squared Error (MSE)** is the average of the squared differences between observed and predicted values. Definitions exist for both the training and test set. The lower the better.

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- **Residual Standard Error (RSE)** is an estimate of the standard deviation of the residuals, giving an idea of the typical size of residuals or “errors.”

$$\text{RSE} = \frac{1}{n - p - 1} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Like them MSE, the lower the better. Unlike MSE, RSE provides a measure of the average error in the *same units as the response variable*, making it easier to interpret.

- R^2 : Indicates the proportion of variance in the dependent variable explained by the independent variables. Higher values indicate better fit but don't necessarily mean a better model, especially if the model is complex.
- **Adjusted R^2** : Adjusts R^2 for the number of predictors, helping avoid overfitting by penalizing for adding variables that don't significantly improve the model. This is useful for comparing models with different numbers of predictors.
- **Root Mean Squared Error (RMSE)**: The average distance between observed and predicted values, in the same units as the response variable. Lower RMSE indicates better fit. Like RSE, RMSE is in the *same units as the response variable*, making it easier to interpret.
- **AIC and BIC**: Lower AIC (Akaike Information Criterion) and BIC (Bayesian Information Criterion) values indicate better model quality, balancing fit with complexity. BIC penalizes complexity more strongly than AIC, which can help avoid overfitting.
- **Mean Absolute Error (MAE)**: The average absolute difference between observed and predicted values, less sensitive to large errors than RMSE.

Validation Approach

The **validation approach** is a technique that splits the data into two sets: the training set (usually 50-80% of the data) and a test set (the rest). The model is fit to the training and the remaining test examples are used to estimate a model's performance. This approach helps assess how well the model generalizes to new, unseen data. We typically look at the MSE, but we could look at any of the metrics discussed above.

Cross-validation Approach

*Cross-validation is a powerful tool to assess the performance of regression models by testing their ability to generalize to new, unseen data. We will learn about this in 571, but for now you can think of cross-validation as a **generalization of the validation approach** that averages multiple evaluations to achieve a more reliable performance estimate.*

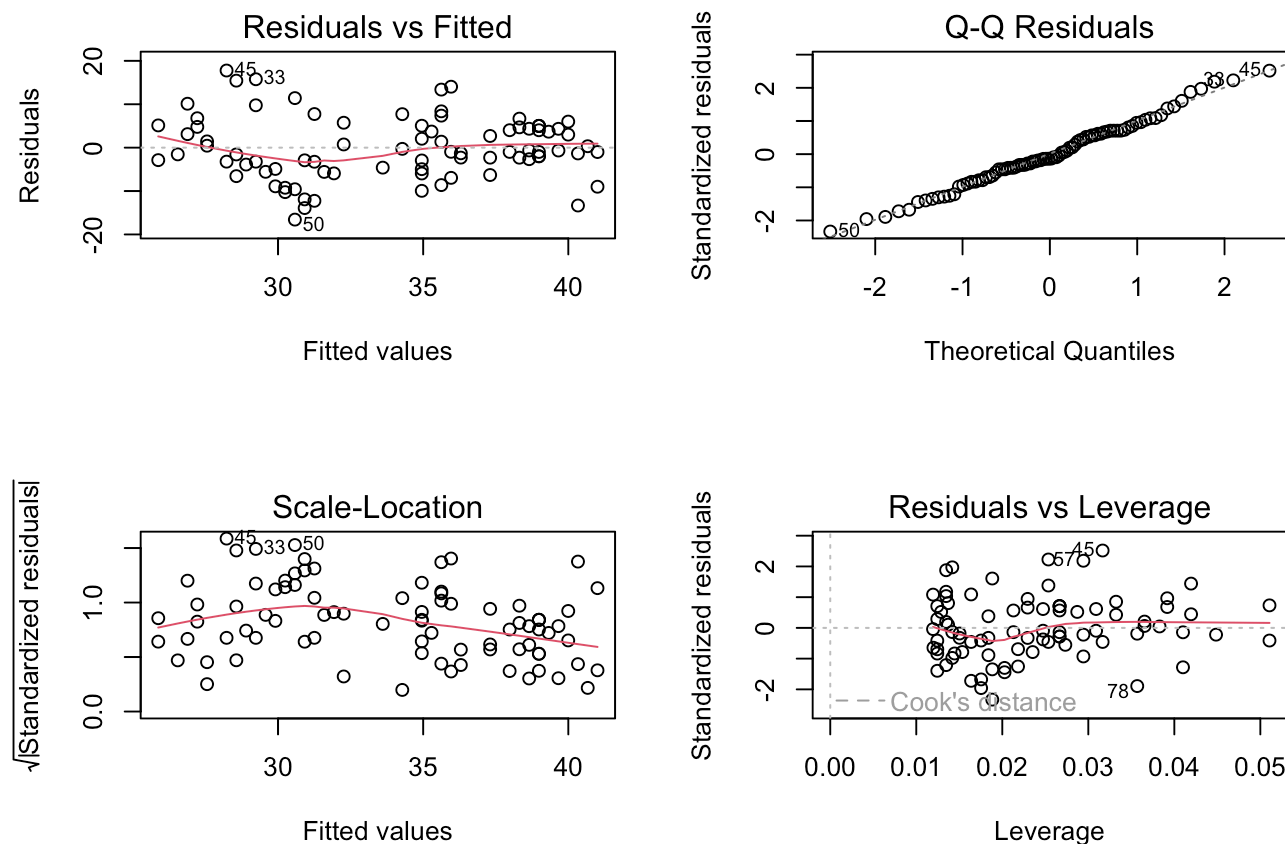
Diagnostics

In addition to the above metrics, several diagnostic checks are important to ensure the model's assumptions hold and to identify any issues that may affect model validity. Here are some common diagnostics for linear regression models:

Residual Analysis

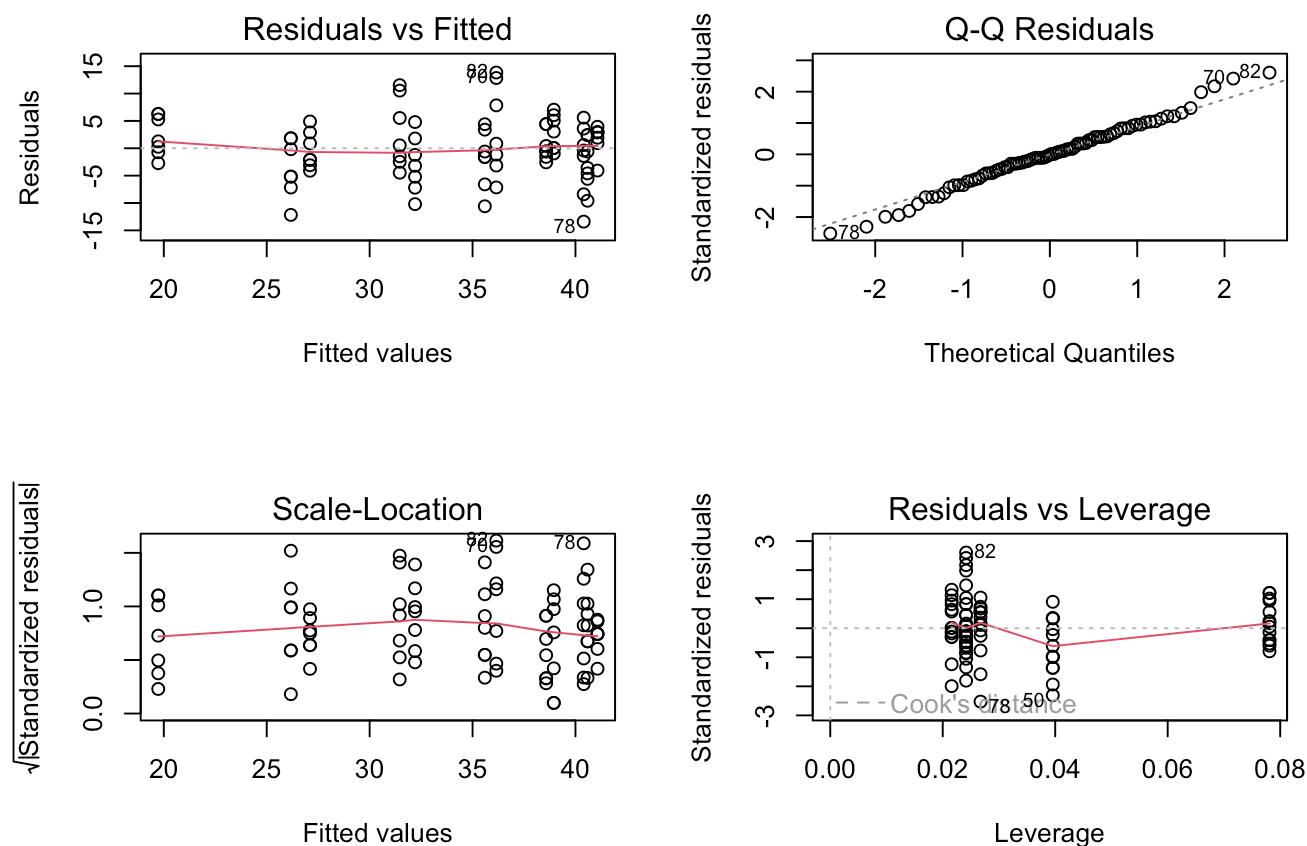
- Plotting residuals (errors) against predicted values or against each predictor helps identify any non-linear relationships, heteroscedasticity, or patterns that violate assumptions.
- Residual Plot:** A scatterplot of residuals versus fitted values. Ideally, residuals should appear randomly scattered around zero, suggesting no systematic pattern.
- Normal Q-Q Plot:** Checks if residuals follow a normal distribution. Points should fall along a straight line if residuals are normally distributed.

```
par(mfrow=c(2,2))
plot(slr)
```



Note that these checks can still work for MLR and polynomial regression:

```
par(mfrow=c(2,2))
plot(order2)
```



Checking for Homoscedasticity

Homoscedasticity implies that the residual variance is constant across all levels of the predictor variables. To check for heteroscedasticity (non-constant variance of residuals) in a regression model, you can use several visual and statistical methods:

Residuals vs. Fitted Values Plot

- Plot the residuals against the predicted (fitted) values. If the residuals exhibit a funnel shape (i.e., they spread out or narrow as fitted values increase), this suggests heteroscedasticity.
- Ideally, the residuals should show a random scatter around zero with a consistent spread across all fitted values.

Scale-Location Plot (Spread-Location Plot)

- This is similar to the residuals vs. fitted plot but uses the square root of the absolute standardized residuals as the y-axis.
- A horizontal line suggests homoscedasticity, while a curved or spread-out pattern suggests heteroscedasticity.

Tip

Sometimes a transformation may help to fix **heteroscedasticity** (e.g. $\log(y)$, $\sqrt{y}$, $\log(x)$). After transformation, always check residuals again!

Normality of Residuals

- Regression assumes that residuals are normally distributed, especially for inference (e.g., hypothesis testing).
- To check the normality of residuals in a regression model, we commonly check the **Q-Q Plot (Quantile-Quantile Plot)**. This is a scatter plot that compares the quantiles of the residuals to the quantiles of a standard normal distribution.
- If the residuals are normally distributed, *the points should lie along a straight diagonal line*. Deviations from this line indicate departures from normality (e.g., a curve in the plot could suggest skewness, and points far from the line could indicate heavy tails).

Other options (that we didn't cover in this module) include:

- **Shapiro-Wilk Test** or **Kolmogorov-Smirnov Test**: Statistical tests to check for normality.
- **Histogram of Residuals**: A visual check of the residual distribution to see if it resembles a normal distribution.

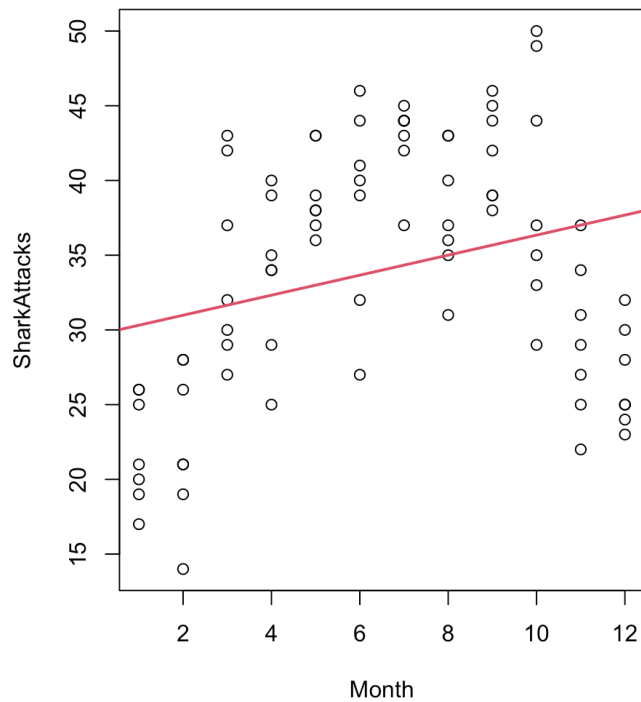
Linearity Check

- The relationship between predictors and the outcome should be linear.
- **Residuals vs. Fitted Plot**: Look for random scatter; patterns might indicate non-linear relationships.

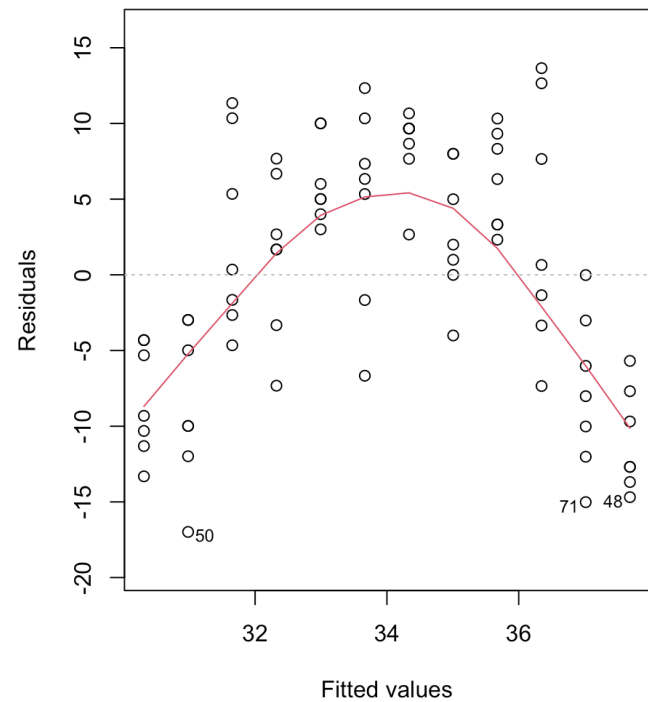
For example, if we tried to find a slr on shark attacks regressed on months, the non-linearity would appear in the residual vs. fitted plot:

► Code

Fitted SLR



Residuals vs Fitted



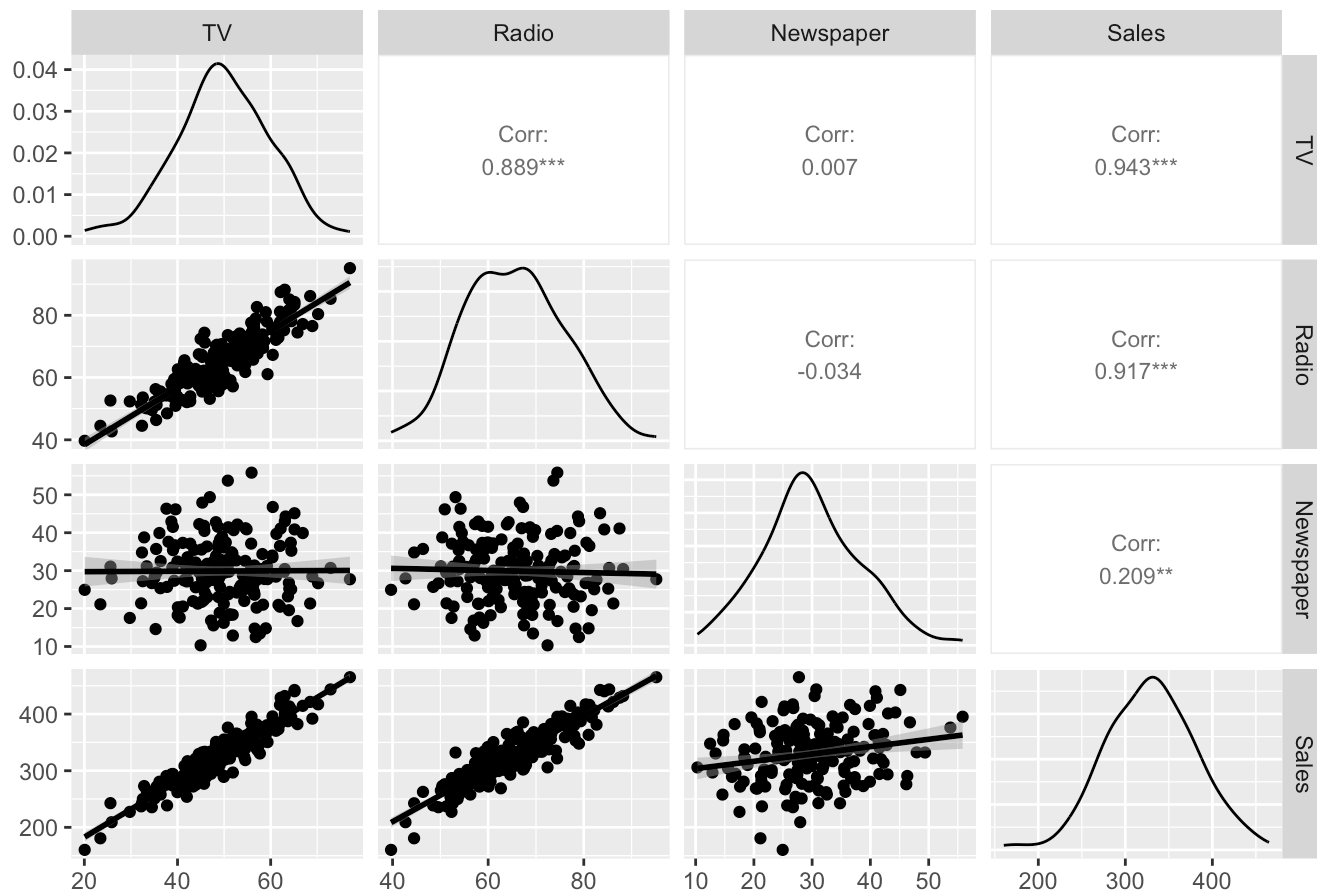
Practical Issues

Multicollinearity Check

- High correlation between predictors can inflate the variance of coefficients, making them unstable.
- **Variance Inflation Factor (VIF)**: VIF values greater than 5 or 10 indicate potential multicollinearity issues.
- **Correlation Matrix**: A quick check for highly correlated predictors.

► Code

Pairwise Scatter Plots and Correlations



Explanation:

- The scatter plots in the lower triangle show the relationships between variables with a smooth trend line.
- The upper triangle displays the Pearson correlation coefficients.
- The diagonal shows the distribution of each variable.

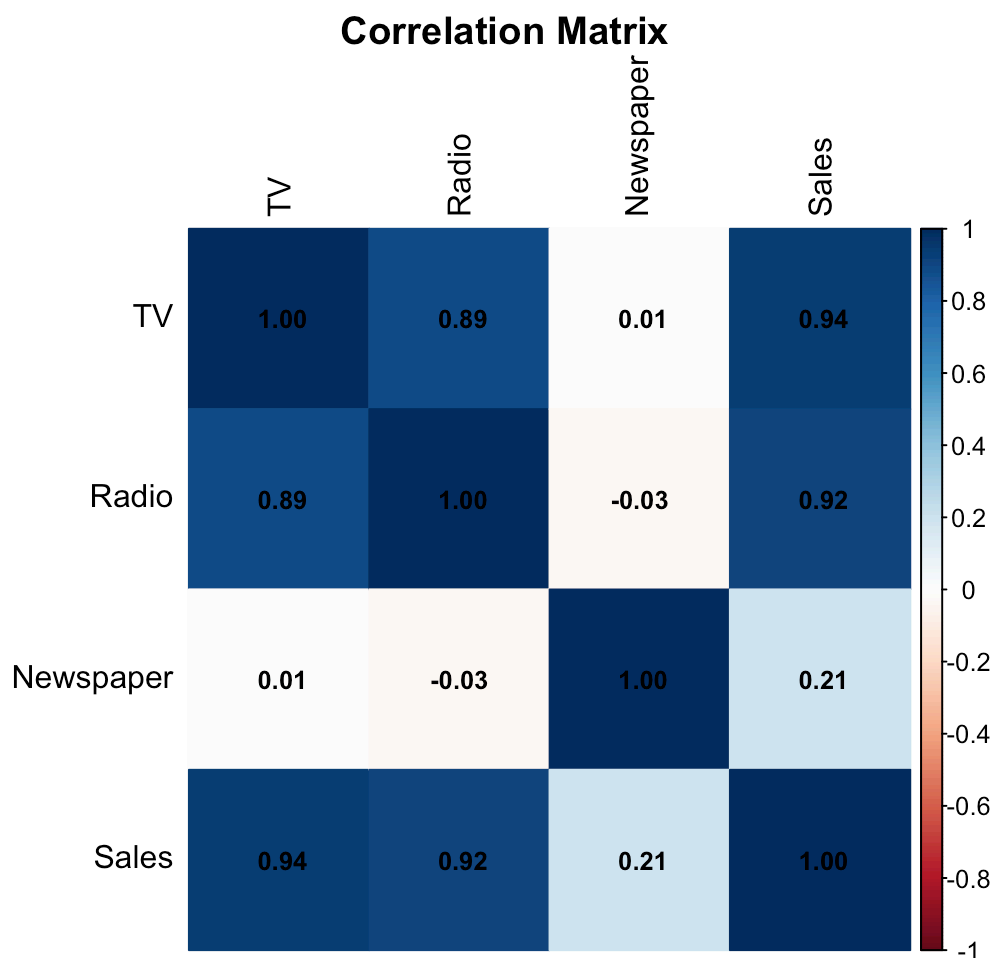
```
# Compute the correlation matrix
cor_matrix <- cor(df)

# Display the correlation matrix
print(cor_matrix)
```

	TV	Radio	Newspaper	Sales
TV	1.00000000	0.88879635	0.00682646	0.9429836
Radio	0.88879635	1.00000000	-0.03368999	0.9166305
Newspaper	0.00682646	-0.03368999	1.00000000	0.2093829
Sales	0.94298361	0.91663045	0.20938289	1.0000000

```
# Visualize the correlation matrix using corrplot
corrplot(cor_matrix, method = "color", addCoef.col = "black",
```

```
tl.col = "black", number.cex = 0.8, title = "Correlation Matrix",
mar = c(0,0,1,0))
```



Interpretation:

- **TV and Radio** have a high correlation (~0.90), indicating multicollinearity.
- **Newspaper** has low correlation with both **TV** and **Radio**.
- **Sales** is strongly correlated with **TV** and **Radio**, and weakly with **Newspaper**.

Let's build a multiple linear regression model to predict **Sales** using all three predictors.

```
# Fit the multiple linear regression model
model <- lm(Sales ~ TV + Radio + Newspaper, data = train_df)

# Summary of the regression model
summary(model)
```

Call:

```
lm(formula = Sales ~ TV + Radio + Newspaper, data = train_df)
```

Residuals:

Min	1Q	Median	3Q	Max
-28.463	-5.553	1.240	6.840	20.544

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	2.9914	7.0050	0.427	0.67
TV	3.0415	0.2075	14.661	<2e-16 ***
Radio	2.0507	0.1986	10.323	<2e-16 ***
Newspaper	1.3671	0.1166	11.729	<2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 9.774 on 104 degrees of freedom

Multiple R-squared: 0.9687, Adjusted R-squared: 0.9678

F-statistic: 1072 on 3 and 104 DF, p-value: < 2.2e-16

Interpretation:

- All predictors are statistically significant ($p < 0.05$).
- **TV** has a large coefficient, but due to high correlation with **Radio**, interpreting individual coefficients becomes problematic.

Variance Inflation Factor (VIF)

To confirm multicollinearity, we'll use the **Variance Inflation Factor (VIF)**. VIF quantifies how much the variance of a regression coefficient is inflated due to multicollinearity.

$$\text{VIF}(X_p) = \frac{1}{1 - R_j^2}$$

where R_j^2 is the coefficient of determination obtained by regressing the j^{th} predictor on all other predictor variables in the model.

Interpreting VIF Results

VIF values help assess the extent of multicollinearity:

- **VIF = 1:** No correlation between the predictor and other variables.
- **1 < VIF < 5:** Moderate correlation; generally acceptable.
- **VIF ≥ 5:** High correlation; indicates potential multicollinearity.
- **VIF ≥ 10:** Severe multicollinearity; problematic for the model.

```
# Calculate VIF for each predictor
vif_values <- vif(model)
vif_values
```

TV	Radio	Newspaper
5.064734	5.045428	1.008708

Interpretation:

- High VIF values for **TV** and **Radio** confirm the presence of multicollinearity;
- **Newspaper** has a small VIF, indicating no multicollinearity.

Dealing with multicollinearity**1. Removing a Predictor (e.g., Radio):**

```
no_multi <- lm(Sales ~ TV + Newspaper, data = train_df)
summary(no_multi)
```

Call:

```
lm(formula = Sales ~ TV + Newspaper, data = train_df)
```

Residuals:

Min	1Q	Median	3Q	Max
-32.280	-8.434	-0.314	10.718	33.822

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	45.8898	7.9859	5.746	9.00e-08 ***
TV	4.9587	0.1309	37.879	< 2e-16 ***
Newspaper	1.3010	0.1648	7.894	2.98e-12 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 13.84 on 105 degrees of freedom

Multiple R-squared: 0.9366, Adjusted R-squared: 0.9354

F-statistic: 775.4 on 2 and 105 DF, p-value: < 2.2e-16

```
vif(no_multi)
```

	TV	Newspaper
	1.005668	1.005668

You can see that this

2. You can combine predictors

```
# Create combined predictor
df$TV_Radio <- df$TV + df$Radio

# Fit combined model
model_combined <- lm(Sales ~ TV_Radio + Newspaper, data = df)
summary(model_combined)
```

Call:

```
lm(formula = Sales ~ TV_Radio + Newspaper, data = df)
```

Residuals:

Min	1Q	Median	3Q	Max
-34.702	-6.768	0.033	6.980	25.457

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	-5.64732	5.13050	-1.101	0.273
TV_Radio	2.54809	0.03747	68.009	<2e-16 ***
Newspaper	1.37822	0.08725	15.797	<2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 9.76 on 177 degrees of freedom

Multiple R-squared: 0.9648, Adjusted R-squared: 0.9644

F-statistic: 2423 on 2 and 177 DF, p-value: < 2.2e-16

```
vif(model_combined)
```

```
TV_Radio Newspaper
1.000199 1.000199
```

3. Use PCA (we'll learn about this later in the program)

4. Apply LASSO or Ridge Regression (we'll learn about this later in the program)

Influential and High Leverage observations

In regression analysis, **high leverage** points and **influential** points both refer to data points that can affect the model's fit, but they have distinct meanings and impacts:

High Leverage Points

- **Definition:** High leverage points are observations with extreme values for one or more predictor variables, meaning they are far from the center of the predictor variable distribution.
- **Effect on Model:** High leverage points have the potential to affect the fit of the regression line because they "pull" the line toward themselves. However, a high leverage point doesn't necessarily change the slope or intercept of the regression line unless it also affects the outcome variable significantly.
- **Detection:** Leverage can be quantified using **leverage values (hat values)**. Observations with leverage values significantly higher than the average (e.g., twice or three times the mean leverage) are considered high leverage points.

Influential Points

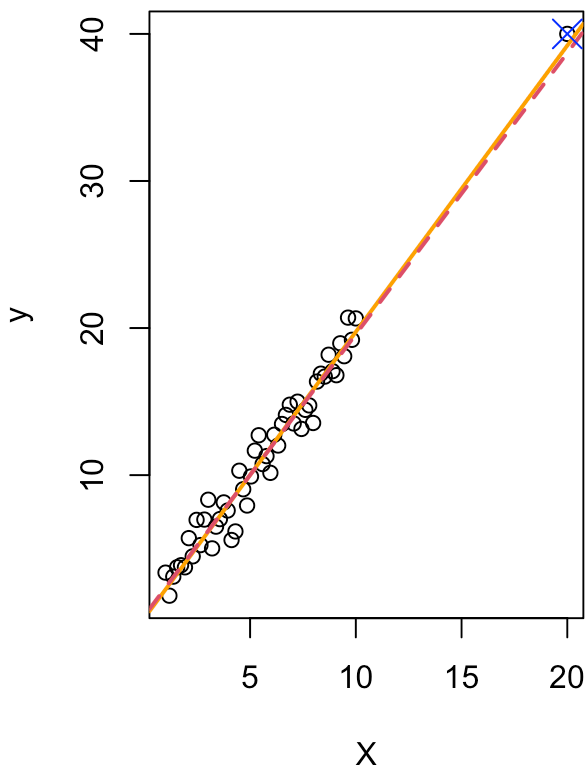
- **Definition:** Influential points are observations that, if removed, would significantly change the slope or intercept of the regression line. In other words, they have a large impact on the regression estimates.
- **Effect on Model:** Influential points affect the overall model fit by altering both the regression coefficients and predictions. They can distort the model's accuracy and lead to biased estimates if not addressed.
- **Detection:** Influential points are commonly detected using **Cook's Distance**, which combines leverage and residual size. Points with high Cook's Distance values are considered influential, typically when they exceed a threshold value like $\frac{4}{n}$, where n is the number of observations.

Key Differences

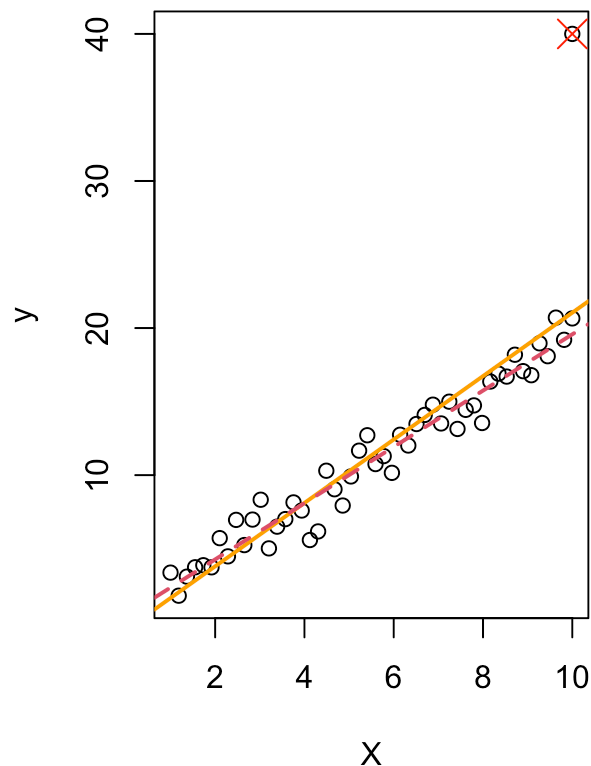
- **Leverage** is a measure of how far an observation's predictor values are from the average predictor values, focusing only on the position in the predictor space.
- **Influence** considers both leverage and residual size, as it depends on how much an observation changes the model's coefficients if removed.

► Code

High Leverage (Not Influential)



Influential Point

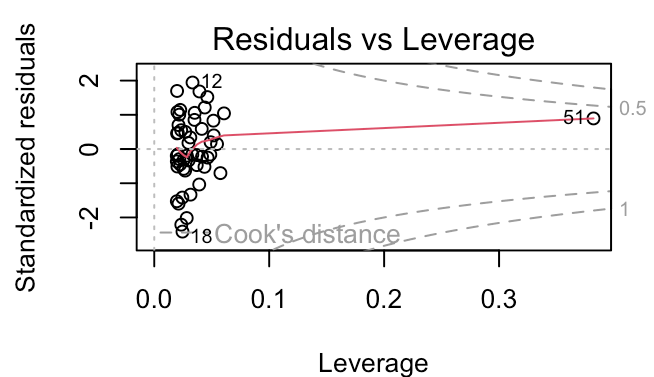
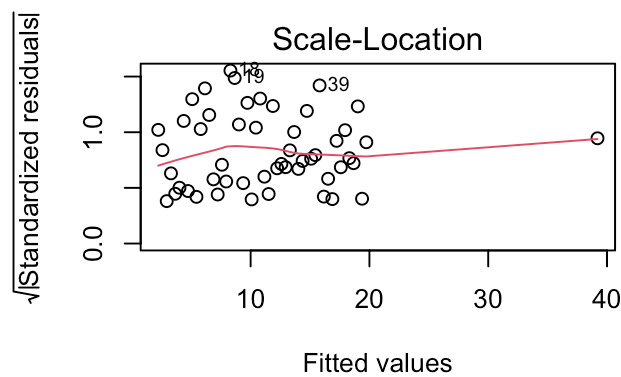
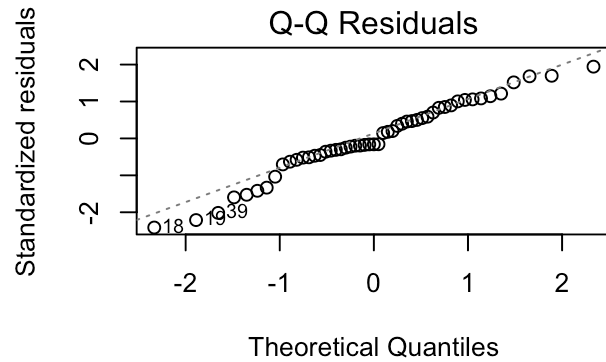
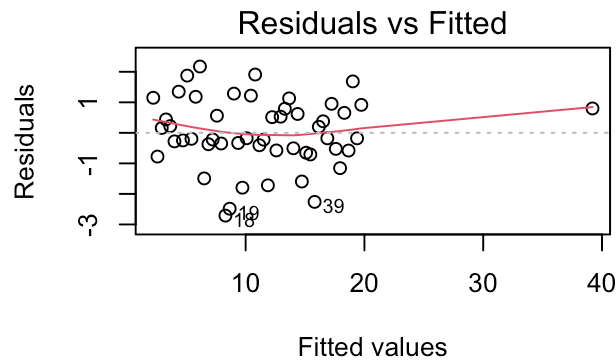


The red dashed line shows the fit of a simply linear regression model that does not include the high leverage and influential point. The orange line indicates the fit with the high leverage point is included (left), and when the influential point was included (right).

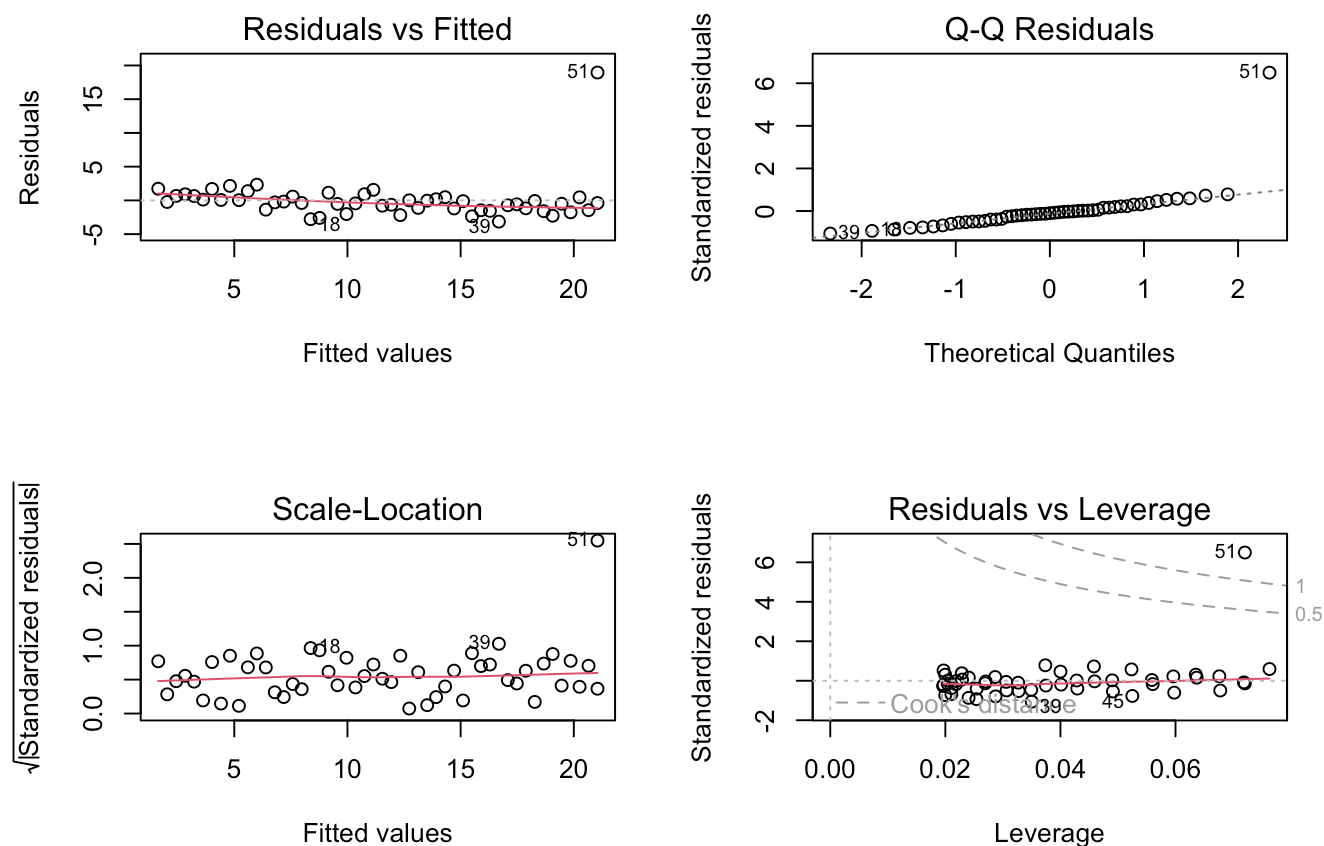
► Code

Both points are apparent in the diagnostic plots:

```
par(mfrow=c(2,2))
plot(model_high_leverage)
```



```
par(mfrow=c(2,2))
plot(model_influential)
```



Sub-optimal models

Overfitting

We can overfit when we use a more complicated model than is needed and we begin to model the noise instead of the underlying relationships. For example

- Selecting too high an M in polynomial regression
- Including unnecessary interaction terms
- Including useless predictors
- Choose an k in KNN that is too small

When we fall into the overfitting category, our models won't generalize to unseen data. A tell-tail sign of this is that our MSE on our training set is very low but the MSE on our test set is high.

Underfitting

Underfitting occurs when a model is too simple to capture the underlying patterns in the data. Some examples of how this might happen

- Selecting too low an M in polynomial regression (linear when there is curvature in the data)

- Failing to Include necessary interaction terms
- Failing to Include usefull predictors
- Choose an k in KNN that is too large

An underfit model has high bias, which means it makes strong assumptions that may not hold in reality. This results in poor performance on both the training data and new (test) data because the model fails to capture important relationships.